HAVE YOU HEARD OF SANTRICITY SNAPSHOTS, LARRY?

Groupings of repos volumes

The main characteristic of these is they're all "repos_"-named volumes for Copy-on-Write data evacuation. A volume may have multiple groups of repos-volumes as a new "grouping" is created for each distinct snapshot "chain". These groupings are realy "Repo Groups", officially misnamed "Snapshot Groups".

	Reserved Capacity For	Status	Associated Base Volume	Snapshot Images	Reserved Capacity Volumes
+	Snapshot Group r1_4096_128_SG_01	Optimal	r1_4096_128	1	1
+	Snapshot Group r1_4096_128_SG_02	Optimal	r1_4096_128	1	2
+	Snapshot Consistency Group bcache Member Volume caching	Optimal	caching	1	1
+	Snapshot Consistency Group bcache Member Volume backing	Optimal	backing	1	1
+	Snapshot Volume bcache_linked_clone_readwrite:caching	Optimal	caching	N/A	1
+	Snapshot Volume bcache_linked_clone_readwrite:backing	Optimal	backing	N/A	1
+	Snapshot Volume r1_4096_128_linked_clone_rw	Optimal	r1_4096_128	N/A	1

*Snapshot Groups are **not** groups of snapshots. They are groups of Repos which are used for CoW data evacuation. R-W Linked Clones also use Repo Groups.*

Based on use case and object name, UI shows groups for **Snapshots, CG Snapshots, and (writable) Linked Clones**.

Repos group has at least 1 "repos" volume(s)

These groups start with one "repos_" volume, but may have two or more of them, even on a different storage pool as Repo Group capacity is added by concatenating, not growing in place, repos volumes. It is easy to see these groupings are not of snapshots, but of repos volumes. Repos volumes may cause waste when oversized or unused.

	Reserved Capacity For	Status	Associated Base Volume	Snapshot Images	Reserved Capacity Volumes
+	Snapshot Group r1_4096_128_SG_01	Optimal	r1_4096_128	1	1
−	Snapshot Group r1_4096_128_SG_02	Optimal	r1_4096_128	1	2

Reserved Capacity Volume Name	Status	Capacity (GiB)	Poo Volu
repos_0003	Optimal	1.50	data
repos_0007	Optimal	4.00	data

+	Snapshot Consistency Group bcache Member Volume caching	Optimal	caching	1	1

scaleoutsean.github.io | CC BY 4.0

HAVE YOU HEARD OF SANTRICITY SNAPSHOTS, LARRY?

Repo groups and snapshot schedules

Schedule Created For	State	Frequency	Start Date
Volume r1_4096_128	Active	Daily/Weekly	Mar 26, 2026
Volume r1_4096_128	Active	Daily/Weekly	Mar 26, 2026

Repo Groups can be created manually ("user-created") or semi-automatically (snapshot schedule-created).
User-created: user creates the first snapshot of a volume. If a Repo Group doesn't exist, it will be created (on the same Disk Group, or a DDP if available & preferred).
Snapshot schedule-created: the same, except that each snapshot schedule will result in separate Repo Group whereas Repo Group created for the first volume snapshot is one per volume. The UI hides the ability to manually create multiple Repo Groups because the maximum number per volume is limited and all this is confusing to most users in any case.

User-created snapshots in multi-repo group situations

Create Snapshot Image ✕

There are multiple instances of reserved capacity associated with Volume **r1_4096_128**. Therefore, you must select the reserved capacity instance on which you want to create the snapshot image.

What is reserved capacity?

Select reserved capacity...

Total Reserved Capacity (GiB)	Available Reserved Capacity (GiB)	Snapshot Images	Snapshot Image Limit
2.50	2.48 (99%)	1	2
5.50	5.48 (99%)	1	1

Create Cancel

Scenario: two schedules for same volume, resulting in two repo groups. User now wants to create a manual snapshot before attempting a database upgrade.

Constraint: one of the repo groups is at, or near, the snapshot count limit.

How to choose? Clearly, we can't even use the second repo group because it was created with a low maximum snapshot count (precisely to avoid random use).

We have no other choice but to use the first group where we're one snapshot below the limit. Is that good? Not necessarily. Once the Nth snapshot is taken (where N is the maximum snapshot count for the Repo Group), no new snapshots may be taken. That means if we create and not delete this additional snapshot, next time this snapshot schedule runs snapshot will fail.
Groups should be selected based on both available capacity and available snapshot count as well as the planned retention for the snapshot.

SANtricity CoW snapshots must be deleted in descending age order, so taking a sapshot in the group with available snapshots would mean we'd have to delete the oldest snapshot before we could delete our temporary snapshot.

scaleoutsean.github.io | CC BY 4.0

HAVE YOU HEARD OF SANTRICITY SNAPSHOTS, LARRY?

Timestamp	Status	Associated Base Volume	Associated Snapshot Consistency Group	Associated Snapshot Volumes
Mar 19, 2026 1:52:01 PM	Optimal	caching	bcache	2
Mar 19, 2026 1:52:01 PM	Optimal	backing	bcache	2
Mar 26, 2026 5:06:14 AM	Optimal	r1_4096_128	None	2
Mar 26, 2026 5:06:50 AM	Optimal	r1_4096_128	None	None

Name	Total Member Volumes	Snapshot Image Limit
− bcache	2	2

Remove

Member Volume Name	Status	Snapshot Images	Snapshot Volumes
caching	Optimal	1	2
backing	Optimal	1	2

Name	Status	Reported Capacity (GiB)	Associated Base Volume	Associated Snapshot Consistency Group
bcache_SV_0001:caching	Optimal	10.00	caching	bcache
bcache_SV_0001:backing	Optimal	200.00	backing	bcache
bcache_linked_clone_readwrite:caching	Optimal	10.00	caching	bcache
bcache_linked_clone_readwrite:backing	Optimal	200.00	backing	bcache
r1_4096_128_linked_clone_rw	Optimal	10.00	r1_4096_128	None
r1_4096_128_linked_clone_ro	Optimal	10.00	r1_4096_128	None

Notice one peculiar possibility: my CG "bcache" has a small "caching" and a much larger "backing" volume. Although these are members of the same Consistency Group, it is likely that, when increasing Repos Group capacity for this CG, I'd probably add capacity to just one member's Repo Group at a time. (Depending on bcache configuration, I may not need to snapshot the caching volume or even have a CG, of course.)

Snapshot Images

Elsewhere known as .... "snapshots". May belong to a Consistency Group. Always read-only.

Consistency Groups

Snapshots of volumes with a common Consistency Group Reference/ID. This shows two member volumes (caching & backing) that belong to a CG named "bcache".
Individual snapshots don't have a cgRef.

Linked Clones

Elegantly misnamed "Snapshot Volumes", these are what most people call Linked Clones and can be read-only or read-write, created from individual volumes or groups.
"Linked" because it's a chain (insert "series of chains" meme).
Read-only Linked Clones volumes don't have own repos volume(s), as there's no need to CoW anything.
A read-write Linked Clone volume (or CG snapshot member) has at least one repos volume as its "chain of snapshots" continues from the Linked Clone volume's base snapshot.

scaleoutsean.github.io | CC BY 4.0

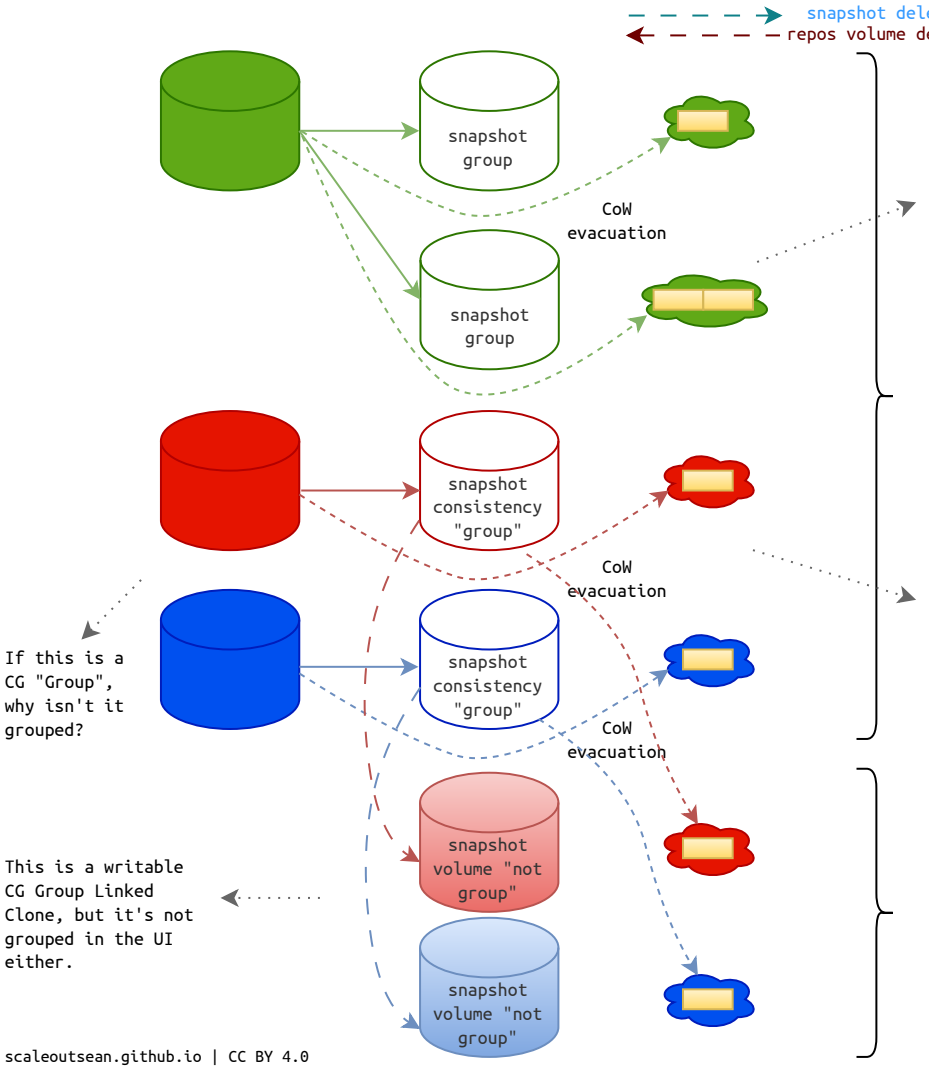
HAVE YOU HEARD OF SANTRICITY SNAPSHOTS, LARRY?

SANtricity "Reserved Capacity" mentions "groups" everywhere. But, what's not a group is called a group, and what is a group is not grouped.

SUGGESTED TERMS AND CONCEPTS
- Repo (or Repos) Group (not Snapshot Group) - (includes repo groups for CG snapshots and writable CG Linked Clones)
- Snapshot, CG Snapshot
- Linked Clone and CG Linked Clone

Volume

SANtricity's confusing "Reserved Capacity" concept



Repos groups have one or more concatenated repos volumes. This one has two. It has nothing to do with Snapshot "Group", CG "Group", volume s "group" (VG), or (CG) Snapshot Volumes. "Repo Group" is a thing. If it can't be fully abstracted, it shouldn't be "hidden".

Repos for snapshot use (both for invididual volumes, and Consistency Groups (ironically, not grouped at all)

The red and blue volume are supposedly a "group", but the UI does not group them. If we ignore the UI and focus on Repos Groups (each volume has its own), that's obvious and we just consider cgRef (CG group ID) to logically "group" them if we need to (maybe we don't as each Repo Group may have different capacity requirements)

Repos for writable "Snapshot Volumes" (also never grouped since every CG member is individually listed) use CoW evacuation zones based on snapshots as CoW "base". When a write to a Linked Clone happens, CoW stores overwritten PIT (snapshot image) data to a Linked Clone member's repo group volume

Repos Groups and Repos Volumes

It is much easier to simply focus on Repos Groups regardless of use case and remember:
- a volume can have several Repos Groups
- a Repos Group is a collection of concatenated repos volumes, although it starts with just one
- due to strict deletion order of snapshots (oldest first) and repos group members (newest first), it is wise to aim for two volume members per each Repos Group
- the UI hides repos volumes, but that is key to avoiding getting confused by the UI or API
- Read-only Linked clones don't write to repos

scaleoutsean.github.io | CC BY 4.0

HAVE YOU HEARD OF SANTRICITY SNAPSHOTS, LARRY?

Once you free your mind of "Snapshot Groups" and start thinking in terms of Repo Groups...

	Day 0	Day 1-8	Day 9	Outcome
SANtricity suggests a "safe" Repo Group capacity amounting to 40% of Base Volume Capacity. That is very likely going to be either **too wasteful, or not enough**. Reject that.	Base Volume Repo Group			
For high-churn volumes, create a "test" daily schedule (20% large) to see how much it consumes. Or two test daily schedules, but two Repo Groups will be created in that case.	20%		20% 20%	You didn't outsmart SANtricity snapshot wizard - you've added 20% and haven't saved anything, but you know your requirement and can confidently create a "production" snapshot schedule.
For low-churn volumes, create a Repo Group amounting to 10% of Base Volume capacity. It can even be a schedule or two if you want to observe consumtion by schedule/pattern.	10%		10% 10%	You saved 20%. Not bad for a week of periodic checking and one adjustment. No harm in having two smaller repos volumes and growing in smaller increments.
For any writable Linked Clone use case, create, observe and expand Linked Clone Repo Group. 5% of Base Volume size may be enough for Linked Clone that lives up to 48h.		48h 5% read-write Linked Clone	5%	4% was needed by a dev/test Linked Clone clone that lives less than two days.

Long-term approach: monitor all Repo Groups, not just those used by snapshots. Use SANtricity snapshots only for short-term use cases; free SANtricity client libraries and automation make it easy to clone, backup and delete Linked Clone - whether it's done with bare metal hosts, Docker containers, or Kubernetes CSI environments. I suggest to not create hourly snapshot schedules that retain more than two snapshots (set Hourly schedule to max out at three and use just two). For Daily, use just one - for backup.

Repo Group monitoring can be done in the API, various SANtricity client libraries and soon E-Series SANtricity Collector and E-Series Performance Analyzer.

scaleoutsean.github.io | CC BY 4.0

HAVE YOU HEARD OF SANTRICITY SNAPSHOTS, LARRY?

GET /volumes

volumeRef ('02')

```
name: r1_4096_128
volumeRef: 020000006D039EA000493A2600000A0669AF8E91
volumeGroupRef: 040000006D039EA000493A9C00000AFD69A67726 # this is Storage Pool Ref, not CG Ref!

name: caching
id:020000006D039EA000493A2600000A1E69B807CE
volumeGroupRef: 040000006D039EA000493A9C00000AFD69A67726

name: backing
id: 020000006D039EA000493A9C00000BF869B80A68
volumeGroupRef: 040000006D039EA000493A9C00000AFD69A67726
```

"caching" and "backing" are in Consistency Group called "bcache"

GET /snapshot-groups

pitGroupRef ('33')

```
name:bcache:caching
pitGroupRef:330000006D039EA000493A9C00000C1C69BBDBC3
baseVolume:020000006D039EA000493A2600000A1E69B807CE
consistencyGroup:True
consistencyGroupRef:2A0000006D039EA000493A2600870A3D69BBD924
repositoryVolume:360000006D039EA000493A9C00000C1B69BBDBC3

name:bcache:backing
pitGroupRef:330000006D039EA000493A9C00000C1F69BBDBC4
baseVolume:020000006D039EA000493A9C00000BF869B80A68
consistencyGroup:True
consistencyGroupRef:2A0000006D039EA000493A2600870A3D69BBD924
repositoryVolume:360000006D039EA000493A9C00000C1E69BBDBC4

name:r1_4096_128_SG_01
pitGroupRef:330000006D039EA000493A2600000A8769C498C1
baseVolume:020000006D039EA000493A2600000A0669AF8E91
consistencyGroup:False
consistencyGroupRef:00000000000000000000000000000000000000
repositoryVolume:360000006D039EA000493A2600000A8669C498C1

name:r1_4096_128_SG_02
pitGroupRef:330000006D039EA000493A2600000A8D69C498E5
baseVolume:020000006D039EA000493A2600000A0669AF8E91
consistencyGroup:False
consistencyGroupRef:00000000000000000000000000000000000000
repositoryVolume:360000006D039EA000493A2600000A8C69C498E5
```

Consistent group of CoWs

CoW volumes

GET /snapshot-images

pitRef ('34')

```
pitRef:340000006D039EA000493A9C00630C2169BBDBE1
pitGroupRef:330000006D039EA000493A9C00000C1C69BBDBC3
baseVol:020000006D039EA000493A2600000A1E69B807CE
consistencyGroupId:2A0000006D039EA000493A2600870A3D69BBD924

pitRef:340000006D039EA000493A9C00630C2269BBDBE1
pitGroupRef:330000006D039EA000493A9C00000C1F69BBDBC4
baseVol:020000006D039EA000493A9C00000BF869B80A68
consistencyGroupId:2A0000006D039EA000493A2600870A3D69BBD924

pitRef:340000006D039EA000493A2600630A8A69C498C2
pitGroupRef:330000006D039EA000493A2600000A8769C498C1
baseVol:020000006D039EA000493A2600000A0669AF8E91
consistencyGroupId:None

pitRef:340000006D039EA000493A2600630A9069C498E5
pitGroupRef:330000006D039EA000493A2600000A8D69C498E5
baseVol:020000006D039EA000493A2600000A0669AF8E91
consistencyGroupId:None
```

GET /consistency-groups

cgRef ('2A')

```
name:bcache
cgRef:2A0000006D039EA000493A2600870A3D69BBD924
```

GET /volumes # where name = repos_*

volumeRef

```
Mapping: repos_<digit> -> concatGroupRef -> pitGroupRef -> volume name
Example: concat volume 360000006D039EA000493A2600000A8669C498C1 -> baseObjectId 330000006D039EA000493A2600000A8769C498C1 ->
pitGroupRef 330000006D039EA000493A2600000A8769C498C1 -> snapshot group for "backing" (part of bcache CG snapshot group)
```

volume CoW

```
repos_0001 volume -> used by volume "caching"
repos_0002 volume -> used by volume "backing"
repos_0000 volume -> repo group for "r1_4096_128" (snapshot schedule one)
repos_0003, repos_0007 volumes -> two-member repos group for "r1_4096_128" (schedule two)
```

CoW repos for snapshots

clone CoW

```
repos_0005 volume -> used for write redirection from snapshot of volume caching in CG bcache_linked_clone_readwrite (r-w Clone)
repos_0004 volume -> used for write redirection from snapshot of volume backing in CG bcache_linked_clone_readwrite (r-w Clone)
repos_0006 volume -> used for write redirection from snapshot of volume r1_4096_128 (read-write Linked Clone)
```

CoW repos for Linked Clones

GET /snapshot-schedules

schedRef

```
schedRef:320000006D039EA000493A26005F0A8969C498C1
action:newpit
targetObject:330000006D039EA000493A2600000A8769C498C1
scheduleStatus:active

schedRef:320000006D039EA000493A26005F0A8F69C498E5
action:newpit
targetObject:330000006D039EA000493A2600000A8D69C498E5
scheduleStatus:active
```

group of CoWs

volume "r1_4096_128" has two snapshot schedules. Each creates own "CoW chain" and uses own group of "repos".
User-created snapshot for a volume can be placed in either group of repos (depending on free capacity, snapshot count limit on the group, etc.). In a group of repos, only the oldest in the chain of CoW snapshots can be deleted, so it isn't always better to use repo group with most space. It is also recommended to limit number of snapshots on each Repo Group when creating a schedule or manually creating a Repo Group.

GET /snapshot-volumes

pitViewRef ('35')

```
name:bcache_SV_0001:caching
viewRef:350000006D039EA000493A9C00000C2569BBDBFC
objectType:pitView
baseVol:020000006D039EA000493A2600000A1E69B807CE
basePIT:340000006D039EA000493A9C00630C2169BBDBE1
accessMode:readOnly
repositoryVolume:0000000000000000000000000000000000000000
membership_viewType:member
membership_cgViewRef:2C0000006D039EA000493A9C002C0C2369BBDBFC

name:bcache_SV_0001:backing
viewRef:350000006D039EA000493A9C00000C2769BBDBFC
objectType:pitView
baseVol:020000006D039EA000493A9C00000BF869B80A68
basePIT:340000006D039EA000493A9C00630C2269BBDBE1
accessMode:readOnly
repositoryVolume:0000000000000000000000000000000000000000
membership_viewType:member
membership_cgViewRef:2C0000006D039EA000493A9C002C0C2369BBDBFC

name:bcache_linked_clone_readwrite:caching
viewRef:350000006D039EA000493A2600000A9469C49921
objectType:pitView
baseVol:020000006D039EA000493A2600000A1E69B807CE
basePIT:340000006D039EA000493A9C00630C2169BBDBE1
accessMode:readWrite
repositoryVolume:360000006D039EA000493A2600000A9369C49921
membership_viewType:member
membership_cgViewRef:2C0000006D039EA000493A26002C0A9269C49921

name:bcache_linked_clone_readwrite:backing
viewRef:350000006D039EA000493A2600000A9769C49921
objectType:pitView
baseVol:020000006D039EA000493A9C00000BF869B80A68
basePIT:340000006D039EA000493A9C00630C2269BBDBE1
accessMode:readWrite
repositoryVolume:360000006D039EA000493A2600000A9669C49921
membership_viewType:member
membership_cgViewRef:2C0000006D039EA000493A26002C0A9269C49921

name:r1_4096_128_linked_clone_rw
viewRef:350000006D039EA000493A2600000A9C69C4995F
objectType:pitView
baseVol:020000006D039EA000493A2600000A0669AF8E91
basePIT:340000006D039EA000493A2600630A8A69C498C2
accessMode:readWrite
repositoryVolume:360000006D039EA000493A2600000A9B69C4995F
membership_viewType:individual
membership_cgViewRef:None

name:r1_4096_128_linked_clone_ro
viewRef:350000006D039EA000493A2600000A9E69C49972
objectType:pitView
baseVol:020000006D039EA000493A2600000A0669AF8E91
basePIT:340000006D039EA000493A2600630A8A69C498C2
accessMode:readOnly
repositoryVolume:0000000000000000000000000000000000000000
membership_viewType:individual
membership_cgViewRef:None
```

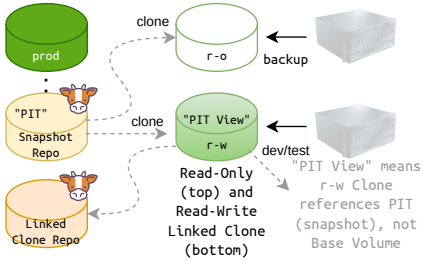
r-o | r-o | r-w | r-w | r-w | r-o

Notice that SANtricity Web UI shows Snapshots, CG Snapshots in separate tabs in Snapshots section, but not in Pools > Reserved Capacity (which also shows repos groups for Linked Clones)!

GET /repositories/concat

concatVolRef ('36')

```
concatVolRef:360000006D039EA000493A9C00000C1B69BBDBC3
memberRefs:020000006D039EA000493A2600000A3E69BBD925
baseObjectType:snapshotGroup
baseObjectId:330000006D039EA000493A9C00000C1C69BBDBC3
memberCount:1
memberNames:repos_0001

concatVolRef:360000006D039EA000493A9C00000C1E69BBDBC4
memberRefs:020000006D039EA000493A9C00000C1A69BBDBC3
baseObjectType:snapshotGroup
baseObjectId:330000006D039EA000493A9C00000C1F69BBDBC4
memberCount:1
memberNames:repos_0002

concatVolRef:360000006D039EA000493A2600000A8669C498C1
memberRefs:020000006D039EA000493A2600000A8469C498C0
baseObjectType:snapshotGroup
baseObjectId:330000006D039EA000493A2600000A8769C498C1
memberCount:1
memberNames:repos_0000

concatVolRef:360000006D039EA000493A2600000A8C69C498E5
memberRefs:020000006D039EA000493A2600000A8B69C498E5,
020000006D039EA000493A2600000A9F69C49A82
baseObjectType:snapshotGroup
baseObjectId:330000006D039EA000493A2600000A8D69C498E5
memberCount:2
memberNames:repos_0003, repos_0007

concatVolRef:360000006D039EA000493A2600000A9369C49921
memberRefs:020000006D039EA000493A9C00000C9669C49BC0
baseObjectType:pitView
baseObjectId:350000006D039EA000493A2600000A9469C49921
memberCount:1
memberNames:repos_0005

concatVolRef:360000006D039EA000493A2600000A9669C49921
memberRefs:020000006D039EA000493A2600000A9169C4991F
baseObjectType:pitView
baseObjectId:350000006D039EA000493A2600000A9769C49921
memberCount:1
memberNames:repos_0004

concatVolRef:360000006D039EA000493A2600000A9B69C4995F
memberRefs:020000006D039EA000493A2600000A9A69C4995E
baseObjectType:pitView
baseObjectId:350000006D039EA000493A2600000A9C69C4995F
memberCount:1
memberNames:repos_0006
```



scaleoutsean.github.io | CC BY 4.0